ParkEase

Build the whole app step by step

A beginner guide to Express, EJS, MySQL, sessions and login/sign up, written for your parking project. Every file is shown in full with a plain explanation.

- Setting up the project and the `.env` file
- Creating the MySQL database and tables
- Writing `server.js` section by section
- EJS syntax: `<%= %>`, `<%- %>`, `<% %>`, include, if, loops
- Sign up, log in, log out, sessions and protected pages
- Redirects, form errors, 404 page and error handling
- All the views: login, signup, dashboard, parking details, profile

How to use this guide

Work from top to bottom. Each step tells you **which file** to open and gives you the code to type in. Type the code yourself instead of copying when you can - that is how it sticks.

Code boxes have a green bar on top with the file name. When a file is split into several boxes (like `server.js`), put the boxes in the file **in the same order** they appear here.

The big picture

When someone opens a page in the browser, this is what happens:

- **1. Browser** asks for a page, for example `GET /dashboard`.
- **2. Express (server.js)** finds the route that matches `/dashboard`.
- **3. Middleware** runs first. `isAuthenticated` checks: is this person logged in? If not, it redirects to `/login`.
- **4. The route** asks **MySQL** for data (parking lots, spaces, user details).
- **5.** `res.render()` sends that data into an **EJS file**, which turns it into HTML.
- **6. Browser** shows the HTML page, styled by `public/css/style.css`.

Final folder structure

```
parking/
|-- .env                 <- secret settings (database password etc.)
|-- database.sql         <- creates the database and tables (NEW)
|-- package.json
|-- server.js            <- the whole backend
|-- public/
|   |-- css/style.css    <- you already have this
|   |-- js/script.js
|-- views/
    |-- partials/
    |   |-- header.ejs   <- top of every page + nav bar (NEW)
    |   |-- footer.ejs   <- bottom of every page (NEW)
    |-- login.ejs
    |-- signup.ejs
    |-- dashboard.ejs
    |-- parking-details.ejs
    |-- profile.ejs
    |-- 404.ejs          <- page not found (NEW)
    |-- error.ejs        <- server error page (NEW)
```

Files marked NEW do not exist yet. Create them as you reach them. You can delete `public/css/index.html` - it is empty and not used.

Step 1 - Project setup

Your `package.json` already lists every package we need. Here is what each one does:

Package	What it does
`express`	The web server. Handles routes like `/login` and `/dashboard`.
`ejs`	Lets you write HTML with JavaScript inside it, so pages can show data.
`mysql2`	Talks to the MySQL database.
`bcrypt`	Scrambles (hashes) passwords so they are never saved as plain text.
`express-session`	Remembers who is logged in between page visits.
`dotenv`	Reads secret settings from the `.env` file.
`nodemon`	Restarts the server every time you save a file (only for development).

If the `node_modules` folder is ever missing, run this inside the `parking` folder:

```
npm install
```

Start the server while you work with:

```
npm run dev
```

Then open `http://localhost:3000` in your browser.

The .env file

Your `.env` file is empty. Put these lines in it. The server reads them with `process.env.NAME`, for example `process.env.DB_PASSWORD`.

.env

```
PORT=3000
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=
DB_NAME=parkease_db
SESSION_SECRET=change_this_to_a_long_random_sentence
```

- No spaces around `=` and no quotes.
- `DB_PASSWORD` stays empty if your MySQL root user has no password (normal on XAMPP). Otherwise put your password.
- `SESSION_SECRET` can be any long sentence. It is used to sign the login cookie so nobody can fake it.

Watch out: Never upload `.env` to GitHub. Create a file called `.gitignore` in the `parking` folder containing the two lines `node_modules` and `.env`.

Step 2 - Create the database

Start MySQL (for example from XAMPP or MySQL Workbench). Create a new file `database.sql` in the `parking` folder, paste the code below, then run it in phpMyAdmin (SQL tab) or MySQL Workbench.

database.sql

```
-- 1. Create the database and start using it
CREATE DATABASE IF NOT EXISTS parkease_db;
USE parkease_db;

-- 2. Users table: one row for every person who signs up
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    phone VARCHAR(20),
    password VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 3. Parking lots table: each parking area
CREATE TABLE parking_lots (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    location VARCHAR(150) NOT NULL,
    price_per_hour DECIMAL(8,2) NOT NULL
);

-- 4. Parking spaces table: the single spots inside a lot
CREATE TABLE parking_spaces (
    id INT AUTO_INCREMENT PRIMARY KEY,
    lot_id INT NOT NULL,
    space_number VARCHAR(10) NOT NULL,
    booked_by INT NULL,
    FOREIGN KEY (lot_id) REFERENCES parking_lots(id),
    FOREIGN KEY (booked_by) REFERENCES users(id)
);

-- 5. Sample data so the dashboard is not empty
INSERT INTO parking_lots (name, location, price_per_hour) VALUES
('City Mall Parking', 'Moi Avenue, Nairobi', 100.00),
('Westlands Plaza', 'Westlands, Nairobi', 150.00),
('Airport Long Stay', 'JKIA, Nairobi', 80.00);

INSERT INTO parking_spaces (lot_id, space_number) VALUES
(1, 'A1'), (1, 'A2'), (1, 'A3'), (1, 'A4'),
(2, 'B1'), (2, 'B2'), (2, 'B3'),
(3, 'C1'), (3, 'C2'), (3, 'C3'), (3, 'C4'), (3, 'C5');
```

What each part means

- `CREATE DATABASE IF NOT EXISTS parkease_db` - makes the database. The name matches `DB_NAME` in `.env`.
- `id INT AUTO_INCREMENT PRIMARY KEY` - every row gets its own number automatically (1, 2, 3...).
- `VARCHAR(100)` - text up to 100 characters. `NOT NULL` means it must be filled in.
- `email ... UNIQUE` - the database refuses two users with the same email.
- `password VARCHAR(255)` - long, because a bcrypt hash is about 60 characters.

- `DECIMAL(8,2)` - money with 2 decimal places, like 150.00.
- `booked_by INT NULL` - holds the id of the user who booked the space. `NULL` means the space is **free**.
- `FOREIGN KEY (lot_id) REFERENCES parking_lots(id)` - a space must belong to a lot that really exists.
- `INSERT INTO ...` - adds sample lots and spaces so you have something to look at.

How the tables connect

Table	Connected to	Meaning
`parking_spaces.lot_id`	`parking_lots.id`	Which lot the space is in
`parking_spaces.booked_by`	`users.id`	Who booked the space (NULL = free)

Step 3 - server.js, section by section

Delete everything in your current `server.js` and rebuild it with the 14 boxes below, in order. Most of it is the same as what you wrote, with a few fixes explained on the way.

3.1 Load the tools

server.js (part 1 of 14)

```
// ===== 1. LOAD THE TOOLS WE NEED =====
require("dotenv").config() // reads the .env file into process.env

const express = require("express")
const mysql = require("mysql2/promise") // the "promise" version lets us use async/await
const bcrypt = require("bcrypt")
const session = require("express-session")

const app = express()
const PORT = process.env.PORT || 3000
```

- `require("dotenv").config()` must be the **first line**, so the `.env` values exist before anything uses them.
- `require("mysql2/promise")` - note the `/promise`. Your old code used plain `mysql2`, which does not work with `await`. That is why `await db.getConnection()` would not have behaved correctly.
- `process.env.PORT || 3000` - use the port from `.env`, or 3000 if it is missing.

3.2 Settings and middleware

Middleware is a function that runs on every request before your route. Think of it as a security guard at the door.

server.js (part 2 of 14)

```
// ===== 2. SETTINGS AND MIDDLEWARE =====
app.set("view engine", "ejs") // render .ejs files from the views folder

app.use(express.static("public")) // serve css, js and images
app.use(express.json()) // read JSON sent to the server
app.use(express.urlencoded({ extended: true })) // read data from HTML forms

app.use(session({
    secret: process.env.SESSION_SECRET || "dev_secret",
    resave: false,
    saveUninitialized: false,
    cookie: {
        httpOnly: true,
        maxAge: 1000 * 60 * 60 // 1 hour
    }
}))

// Make "user" available inside EVERY ejs file (used by the nav bar)
app.use((req, res, next) => {
    res.locals.user = req.session.user || null
    next()
})
```

- `app.set("view engine", "ejs")` - now `res.render("login")` means `views/login.ejs`.

- `express.static("public")` - a link to `/css/style.css` loads `public/css/style.css`.
- `express.urlencoded(...)` - this is what fills `req.body` when a form is submitted. Without it `req.body` is undefined.
- `session(...)` - gives every visitor a cookie. Whatever you put in `req.session` is remembered on their next request.
- `httpOnly: true` - JavaScript in the browser cannot read the cookie (safer).
- `maxAge` - after 1 hour the cookie expires and the user must log in again.
- `res.locals.user` - anything put in `res.locals` is available in **every** EJS file. The nav bar uses it to show Log in or Log out.
- `next()` - means "I am done, go to the next step". If you forget it, the page loads forever.

3.3 Connect to the database

server.js (part 3 of 14)

```
// ===== 3. DATABASE CONNECTION =====
const db = mysql.createPool({
    host: process.env.DB_HOST,
    user: process.env.DB_USER,
    password: process.env.DB_PASSWORD,
    database: process.env.DB_NAME,
    waitForConnections: true,
    connectionLimit: 10
})

async function connectionTest() {
    try {
        const connection = await db.getConnection()
        console.log("Database connected successfully")
        connection.release() // give the connection back to the pool
    } catch (error) {
        console.log("Database connection failed:", error.message)
    }
}
connectionTest()
```

- `createPool` - keeps up to 10 connections open and reuses them. Faster than opening a new one each time.
- The settings come from `.env` instead of being typed in the code.
- `try { } catch (error) { }` - if the code inside `try` fails, the program does not crash. It jumps to `catch` and we print the reason.
- `connection.release()` - this was missing before. Always give a connection back when you are done, or the pool runs out.

Tip: If you see `ECONNREFUSED` in the terminal, MySQL is not running. If you see `Unknown database`, you did not run `database.sql`. If you see `Access denied`, check `DB_USER` and `DB_PASSWORD`.

3.4 Authentication middleware

server.js (part 4 of 14)

```
// ===== 4. AUTH MIDDLEWARE =====
// Stops people who are NOT logged in
function isAuthenticated(req, res, next) {
    if (!req.session.user) {
        return res.redirect("/login")
    }
    next()
}

// Stops people who ARE logged in from seeing login/signup again
function isGuest(req, res, next) {
    if (req.session.user) {
        return res.redirect("/dashboard")
    }
    next()
}
```

- `isAuthenticated` - put it on pages only logged-in users may see. No `req.session.user`? Redirect to `/login`.
- `isGuest` - the opposite. A logged-in user who visits `/login` is sent to `/dashboard`.
- `return res.redirect(...)` - the `return` stops the function, so `next()` never runs.
- You use them by putting them between the path and the route function: `app.get("/dashboard", isAuthenticated, ...)`.

Watch out: In your old code `isAuthenticated` was written but never used, so anyone could open `/dashboard` without logging in.

3.5 Home route

server.js (part 5 of 14)

```
// ===== 5. HOME ROUTE =====
app.get("/", (req, res) => {
    if (req.session.user) {
        return res.redirect("/dashboard")
    }
    res.redirect("/login")
})
```

`/` does not have its own page. It just sends people to the right place: dashboard if logged in, login if not.

3.6 Sign up

server.js (part 6 of 14)

```
// ===== 6. SIGN UP =====
app.get("/signup", isGuest, (req, res) => {
    res.render("signup", { error: null, old: {} })
})

app.post("/signup", isGuest, async (req, res) => {
    const { full_name, email, phone, password, confirm_password } = req.body

    // Check the form first
    if (!full_name || !email || !password) {
        return res.render("signup", { error: "Please fill in all required fields", old: req.body })
    }
    if (password.length < 6) {
        return res.render("signup", { error: "Password must be at least 6 characters", old: req.body })
    }
```

```
    if (password !== confirm_password) {
        return res.render("signup", { error: "Passwords do not match", old: req.body })
    }

    try {
        // Is the email already used?
        const [rows] = await db.query("SELECT id FROM users WHERE email = ?", [email])
        if (rows.length > 0) {
            return res.render("signup", { error: "That email is already registered", old: req.body })
        }

        // Scramble the password before saving it
        const hashedPassword = await bcrypt.hash(password, 10)

        await db.query(
            "INSERT INTO users (full_name, email, phone, password) VALUES (?, ?, ?, ?)",
            [full_name, email, phone, hashedPassword]
        )

        res.redirect("/login?registered=1")
    } catch (error) {
        console.log("Signup error:", error.message)
        res.render("signup", { error: "Something went wrong. Please try again.", old: req.body })
    }
})
```

There are **two** routes for `/signup`:

- `app.get` - shows the empty form. We pass `error: null` and `old: {}` because the EJS file uses them.
- `app.post` - runs when the form is submitted (`<form action="/signup" method="POST">`).
- `const { full_name, email, ... } = req.body` - pulls each form field out by its `name="..."`.
- **Validation**: check empty fields, password length and matching passwords **before** touching the database. On a problem we render the form again with an error message.
- `old: req.body` - sends back what they typed so they do not have to type it all again.
- `const [rows] = await db.query(...)` - `db.query` returns a list; the first item is the rows found.
- `?` placeholders - the values in the array replace the `?` safely. **Never** glue user input into SQL with `+`. That allows SQL injection attacks.
- `bcrypt.hash(password, 10)` - scrambles the password. `10` is how hard it works. You cannot turn a hash back into the password.
- `res.redirect("/login?registered=1")` - after success, go to login. `?registered=1` tells the login page to show a green message.

3.7 Log in

server.js (part 7 of 14)

```
// ===== 7. LOG IN =====
app.get("/login", isGuest, (req, res) => {
    const success = req.query.registered ? "Account created. Please log in." : null
    res.render("login", { error: null, success: success })
})

app.post("/login", isGuest, async (req, res) => {
```

```
    const { email, password } = req.body

    if (!email || !password) {
        return res.render("login", { error: "Enter your email and password", success: null })
    }

    try {
        const [rows] = await db.query("SELECT * FROM users WHERE email = ?", [email])
        const foundUser = rows[0]

        // Same message for both cases so attackers can't guess emails
        if (!foundUser) {
            return res.render("login", { error: "Wrong email or password", success: null })
        }

        const passwordMatches = await bcrypt.compare(password, foundUser.password)
        if (!passwordMatches) {
            return res.render("login", { error: "Wrong email or password", success: null })
        }

        // Save ONLY safe details in the session (never the password)
        req.session.user = {
            id: foundUser.id,
            full_name: foundUser.full_name,
            email: foundUser.email
        }

        res.redirect("/dashboard")
    } catch (error) {
        console.log("Login error:", error.message)
        res.render("login", { error: "Something went wrong. Please try again.", success: null })
    }
})
```

- `req.query.registered` - reads the `?registered=1` part of the URL.
- We look up the user by email. `rows[0]` is the first (and only) user, or `undefined` if none.
- `bcrypt.compare(password, foundUser.password)` - hashes what they typed and compares it to the saved hash. Gives `true` or `false`.
- We show the **same** message for wrong email and wrong password, so nobody can find out which emails are registered.
- `req.session.user = {...}` - **this is the moment the user becomes logged in.** From now on every request has `req.session.user`.
- Never put the password in the session.

3.8 Log out

server.js (part 8 of 14)

```
// ===== 8. LOG OUT =====
app.post("/logout", (req, res) => {
    req.session.destroy(() => {
        res.clearCookie("connect.sid")
        res.redirect("/login")
    })
})
```

- `req.session.destroy()` - deletes the session, so `req.session.user` is gone.

- `res.clearCookie("connect.sid")` - removes the cookie from the browser. `connect.sid` is the default cookie name.
- We use `POST`, not `GET`, so a simple link or image cannot log someone out by accident. The nav bar has a small form for it.

3.9 Dashboard

server.js (part 9 of 14)

```
// ===== 9. DASHBOARD (protected) =====
app.get("/dashboard", isAuthenticated, async (req, res, next) => {
    try {
        const [lots] = await db.query(`
            SELECT parking_lots.*,
                   COUNT(parking_spaces.id) AS total_spaces,
                   SUM(parking_spaces.booked_by IS NULL) AS free_spaces
            FROM parking_lots
            LEFT JOIN parking_spaces ON parking_spaces.lot_id = parking_lots.id
            GROUP BY parking_lots.id
        `)
        res.render("dashboard", { lots: lots })
    } catch (error) {
        next(error) // send the error to the error handler at the bottom
    }
})
```

- `isAuthenticated` protects the page.
- The SQL gets every lot, plus how many spaces it has (`COUNT`) and how many are free (`SUM(booked_by IS NULL)` counts the rows where booked_by is empty).
- `LEFT JOIN` - include lots even if they have no spaces yet.
- The backtick quotes around the SQL (the key left of 1) let one string go over many lines.
- `res.render("dashboard", { lots: lots })` - the EJS file can now use a variable called `lots`.
- `next(error)` - instead of handling the error here, pass it to the error handler (part 13). One place shows all server errors.

3.10 Parking details, book and release

server.js (part 10 of 14)

```
// ===== 10. PARKING DETAILS (protected) =====
app.get("/parking/:id", isAuthenticated, async (req, res, next) => {
    try {
        const lotId = req.params.id

        const [lotRows] = await db.query("SELECT * FROM parking_lots WHERE id = ?", [lotId])
        if (lotRows.length === 0) {
            return res.status(404).render("404")
        }

        const [spaces] = await db.query(
            "SELECT * FROM parking_spaces WHERE lot_id = ? ORDER BY space_number",
            [lotId]
        )

        res.render("parking-details", {
            lot: lotRows[0],
```

```
            spaces: spaces,
            message: req.query.msg || null
        })
    } catch (error) {
        next(error)
    }
})

// Book a free space
app.post("/spaces/:id/book", isAuthenticated, async (req, res, next) => {
    try {
        const spaceId = req.params.id
        const userId = req.session.user.id

        // Only books it if nobody else has it ("booked_by IS NULL")
        const [result] = await db.query(
            "UPDATE parking_spaces SET booked_by = ? WHERE id = ? AND booked_by IS NULL",
            [userId, spaceId]
        )

        const [rows] = await db.query("SELECT lot_id FROM parking_spaces WHERE id = ?", [spaceId])
        if (rows.length === 0) {
            return res.status(404).render("404")
        }

        const msg = result.affectedRows === 1 ? "booked" : "taken"
        res.redirect(`/parking/${rows[0].lot_id}?msg=${msg}`)
    } catch (error) {
        next(error)
    }
})

// Release a space you booked
app.post("/spaces/:id/release", isAuthenticated, async (req, res, next) => {
    try {
        await db.query(
            "UPDATE parking_spaces SET booked_by = NULL WHERE id = ? AND booked_by = ?",
            [req.params.id, req.session.user.id]
        )
        res.redirect("/profile")
    } catch (error) {
        next(error)
    }
})
```

- `/parking/:id` - `:id` is a **route parameter**. Visiting `/parking/2` makes `req.params.id` equal `"2"`. Your old route was just `/parking`, which could not know which lot to show.
- If no lot has that id we show the 404 page with `res.status(404)`.
- `req.query.msg` - reads `?msg=booked` or `?msg=taken` from the URL to show a message.
- **Booking**: `UPDATE ... WHERE id = ? AND booked_by IS NULL` only changes the row if the space is still free. If two people click at the same moment, only one wins.
- `result.affectedRows` - how many rows changed. 1 = booked, 0 = someone was faster.
- **Release**: `AND booked_by = ?` makes sure you can only release a space **you** booked.
- This pattern - POST, change data, then redirect - stops the browser from resubmitting the form if the user refreshes.

3.11 Profile

server.js (part 11 of 14)

```
// ===== 11. PROFILE (protected) =====
app.get("/profile", isAuthenticated, async (req, res, next) => {
    try {
        const [userRows] = await db.query(
            "SELECT full_name, email, phone, created_at FROM users WHERE id = ?",
            [req.session.user.id]
        )

        const [bookings] = await db.query(`
            SELECT parking_spaces.id, parking_spaces.space_number,
                   parking_lots.name AS lot_name, parking_lots.location
            FROM parking_spaces
            JOIN parking_lots ON parking_lots.id = parking_spaces.lot_id
            WHERE parking_spaces.booked_by = ?
        `, [req.session.user.id])

        res.render("profile", { profile: userRows[0], bookings: bookings })
    } catch (error) {
        next(error)
    }
})
```

- We read the user again from the database to get fresh details (phone, created_at).
- We only `SELECT` the columns we need. The password is never sent to the page.
- The second query finds every space where `booked_by` is this user, joined with its lot name.

3.12 404 - page not found

server.js (part 12 of 14)

```
// ===== 12. 404 - PAGE NOT FOUND =====
// Runs only if no route above matched
app.use((req, res) => {
    res.status(404).render("404")
})
```

Express checks routes from top to bottom. If none matched, this one runs. It must come **after** all your routes.

3.13 Error handler (handling exceptions)

server.js (part 13 of 14)

```
// ===== 13. ERROR HANDLER =====
// Express knows this is the error handler because it has 4 inputs
app.use((err, req, res, next) => {
    console.log("Server error:", err.message)
    res.status(500).render("error", { message: "Something went wrong on our side." })
})
```

- An error handler has **4** inputs: `(err, req, res, next)`. That is how Express knows it is special.
- Every `next(error)` in the routes ends up here.
- We log the real error for you in the terminal, but the user only sees a friendly message.
- In Express 5 (your version), errors thrown inside `async` routes also reach this handler automatically. Using `try/catch` + `next(error)` is still clearer for learning.

3.14 Start the server

server.js (part 14 of 14)

```
// ===== 14. START THE SERVER =====
app.listen(PORT, () => {
    console.log(`Server running at http://localhost:${PORT}`)
})
```

`app.listen` starts the server. This is always the last thing in the file.

Summary of routes

Method + path	Who can open it	What it does
GET `/`	anyone	Redirects to dashboard or login
GET / POST `/signup`	guests only	Show form / create the account
GET / POST `/login`	guests only	Show form / log in
POST `/logout`	anyone	End the session
GET `/dashboard`	logged in	List all parking lots
GET `/parking/:id`	logged in	Show one lot and its spaces
POST `/spaces/:id/book`	logged in	Book a free space
POST `/spaces/:id/release`	logged in	Free your own space
GET `/profile`	logged in	Your details and bookings

Step 4 - EJS in 5 minutes

EJS is normal HTML with special tags. Only three tags matter:

Tag	What it does	Example
`<%= %>`	Prints a value, made safe (HTML is escaped). Use this for almost everything.	`<%= user.full_name %>`
`<%- %>`	Prints **raw** HTML. Only use it for `include`.	`<%- include("header") %>`
`<% %>`	Runs JavaScript but prints nothing. Used for `if` and loops.	`<% if (error) { %>`

Showing a variable

If the route does `res.render("login", { error: "Wrong password" })`, then in `login.ejs`:

```
<p><%= error %></p>      <!-- shows: <p>Wrong password</p> -->
```

If / else

```
<% if (user) { %>
    <p>Welcome back!</p>
<% } else { %>
    <p>Please log in.</p>
<% } %>
```

Notice each line of JavaScript has its own `<% %>`, and the HTML sits between them.

Loop over a list

```
<% lots.forEach(function(lot) { %>
    <h3><%= lot.name %></h3>
<% }) %>
```

The `<h3>` is repeated once for every lot. Do not forget the closing `<% }) %>`.

Include another file

```
<%- include("partials/header", { title: "Log in" }) %>
```

Pastes `views/partials/header.ejs` here. The object `{ title: ... }` sends extra variables to it.

Inside an attribute

```
<a href="/parking/<%= lot.id %>">View</a>
<input name="email" value="<%= old.email || '' %>">
```

`|| ''` means: if `old.email` is empty, print nothing instead of the word undefined.

Watch out: If EJS says `X is not defined`, you used a variable the route did not send. Check the object in `res.render()`.

Step 5 - Partials (header and footer)

Every page needs the same `<head>`, CSS link and nav bar. Instead of copying it into each page, we write it once. Create a folder `views/partials` and add these two files.

views/partials/header.ejs

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title><%= title %> | ParkEase</title>
    <link rel="stylesheet" href="/css/style.css">
</head>
<body>
    <nav>
        <a href="/"><strong>ParkEase</strong></a>
        <div>
            <% if (user) { %>
                <a href="/dashboard">Dashboard</a>
                <a href="/profile">Profile</a>
                <form action="/logout" method="POST" style="display:inline; width:auto;">
                    <button type="submit">Log out</button>
                </form>
            <% } else { %>
                <a href="/login">Log in</a>
                <a href="/signup">Sign up</a>
            <% } %>
        </div>
    </nav>
```

- `<%= title %>` - each page sends its own title through `include`.
- `<link rel="stylesheet" href="/css/style.css">` - starts with `/` so it works on every page, even `/parking/2`.
- `<% if (user) { %>` - `user` comes from `res.locals.user` (part 2 of server.js). Logged in users see Dashboard, Profile and Log out; guests see Log in and Sign up.
- Log out is a small `POST` form styled inline so it sits in the nav bar.
- The header **opens** `<html>` and `<body>` but does not close them. The footer closes them.

views/partials/footer.ejs

```
    <script src="/js/script.js"></script>
</body>
</html>
```

Step 6 - signup.ejs

views/signup.ejs

```
<%- include("partials/header", { title: "Sign up" }) %>

<div class="auth-container">
    <h1>Create account</h1>
    <p>Sign up to find and book parking.</p>

    <% if (error) { %>
        <div class="alert-error"><%= error %></div>
    <% } %>

    <form action="/signup" method="POST">
        <label for="full_name">Full name *</label>
        <input type="text" id="full_name" name="full_name"
               value="<%= old.full_name || '' %>" required>

        <label for="email">Email *</label>
        <input type="email" id="email" name="email"
               value="<%= old.email || '' %>" required>

        <label for="phone">Phone</label>
        <input type="tel" id="phone" name="phone"
               value="<%= old.phone || '' %>">

        <label for="password">Password * <small>(at least 6 characters)</small></label>
        <input type="password" id="password" name="password" minlength="6" required>

        <label for="confirm_password">Confirm password *</label>
        <input type="password" id="confirm_password" name="confirm_password" required>

        <button type="submit">Sign up</button>
    </form>

    <p style="margin-top:18px;">Already have an account? <a href="/login">Log in</a></p>
</div>

<%- include("partials/footer") %>
```

- `action="/signup" method="POST"` - sends the form to `app.post("/signup")`.
- `name="full_name"` - must match exactly what the server reads from `req.body`. A typo here means `undefined` on the server.
- `<label for="email">` + `id="email"` - clicking the label focuses the box.
- `required`, `type="email"`, `minlength="6"` - the browser checks these before sending. The server checks again, because browser checks can be skipped.
- `value="<%= old.email || '' %>"` - refills the box after an error. Passwords are never refilled.
- `class="auth-container"` and `class="alert-error"` already exist in your `style.css`.

Step 7 - login.ejs

views/login.ejs

```
<%- include("partials/header", { title: "Log in" }) %>

<div class="auth-container">
    <h1>Welcome back</h1>
    <p>Log in to manage your parking.</p>
```

```
    <% if (success) { %>
        <div class="card" style="padding:12px 15px;"><%= success %></div>
    <% } %>

    <% if (error) { %>
        <div class="alert-error"><%= error %></div>
    <% } %>

    <form action="/login" method="POST">
        <label for="email">Email</label>
        <input type="email" id="email" name="email" required>

        <label for="password">Password</label>
        <input type="password" id="password" name="password" required>

        <button type="submit">Log in</button>
    </form>

    <p style="margin-top:18px;">No account yet? <a href="/signup">Sign up</a></p>
</div>

<%- include("partials/footer") %>
```

- Two messages: green `success` (after signing up) and red `error` (wrong details). The server always sends both, one of them as `null`.
- The form posts to `/login`, which runs `bcrypt.compare` and creates the session.

Step 8 - dashboard.ejs

views/dashboard.ejs

```
<%- include("partials/header", { title: "Dashboard" }) %>

<main>
    <h1>Hello, <%= user.full_name %></h1>
    <p>Pick a parking lot to see its free spaces.</p>

    <% if (lots.length === 0) { %>
        <div class="card">No parking lots yet.</div>
    <% } else { %>
        <div class="lots-grid">
            <% lots.forEach(function(lot) { %>
                <div class="lot-card">
                    <h3><%= lot.name %></h3>
                    <p><%= lot.location %></p>
                    <p>KES <%= lot.price_per_hour %> per hour</p>
                    <p><strong><%= lot.free_spaces || 0 %></strong> of <%= lot.total_spaces %> spaces free<
                    <a class="btn" href="/parking/<%= lot.id %>">View spaces</a>
                </div>
            <% }) %>
        </div>
    <% } %>
</main>

<%- include("partials/footer") %>
```

- `<%= user.full_name %>` - the name saved in the session at login.
- `if (lots.length === 0)` - always handle an empty list, so the page does not look broken.
- `lots.forEach(...)` - one `lot-card` per parking lot. `lots-grid` puts them side by side.
- `lot.free_spaces || 0` - shows 0 if a lot has no spaces yet.
- `href="/parking/<%= lot.id %>"` - builds a link like `/parking/1`, which hits the `/parking/:id` route.

Step 9 - parking-details.ejs

views/parking-details.ejs

```
<%- include("partials/header", { title: lot.name }) %>

<main>
    <a href="/dashboard">&larr; Back to dashboard</a>

    <div class="parking-card" style="margin-top:15px;">
        <h1><%= lot.name %></h1>
        <p><%= lot.location %></p>
        <p>KES <%= lot.price_per_hour %> per hour</p>
    </div>

    <% if (message === "booked") { %>
        <div class="card">Space booked. You can see it on your profile.</div>
    <% } else if (message === "taken") { %>
        <div class="alert-error">Sorry, someone already booked that space.</div>
    <% } %>

    <h2>Spaces</h2>
    <div class="space-grid">
        <% spaces.forEach(function(space) { %>
            <div class="space">
```

```
                <p><%= space.space_number %></p>

                <% if (space.booked_by === null) { %>
                    <form action="/spaces/<%= space.id %>/book" method="POST">
                        <button type="submit">Book</button>
                    </form>
                <% } else if (space.booked_by === user.id) { %>
                    <small>Yours</small>
                <% } else { %>
                    <small>Taken</small>
                <% } %>
            </div>
        <% }) %>
    </div>
</main>

<%- include("partials/footer") %>
```

- `include("partials/header", { title: lot.name })` - the browser tab shows the lot name.
- `message` - comes from `?msg=` in the URL after booking.
- Each space has three possible states, decided with `if / else if / else`:
  - `booked_by === null` - free, so show a **Book** button (a POST form).
  - `booked_by === user.id` - you booked it, show **Yours**.
  - anything else - someone else has it, show **Taken**.
- `←` is the HTML code for a left arrow.

Step 10 - profile.ejs

views/profile.ejs

```
<%- include("partials/header", { title: "Profile" }) %>

<main>
    <h1>My profile</h1>

    <div class="profile-card">
        <p><strong>Name:</strong> <%= profile.full_name %></p>
        <p><strong>Email:</strong> <%= profile.email %></p>
        <p><strong>Phone:</strong> <%= profile.phone || "Not added" %></p>
        <p><strong>Member since:</strong> <%= new Date(profile.created_at).toDateString() %></p>
    </div>

    <h2>My booked spaces</h2>

    <% if (bookings.length === 0) { %>
        <div class="card">You have not booked any space yet.</div>
    <% } else { %>
        <div class="cards-grid">
            <% bookings.forEach(function(booking) { %>
                <div class="card">
                    <h3>Space <%= booking.space_number %></h3>
                    <p><%= booking.lot_name %> - <%= booking.location %></p>
                    <form action="/spaces/<%= booking.id %>/release" method="POST">
                        <button type="submit">Release space</button>
                    </form>
                </div>
            <% }) %>
        </div>
    <% } %>
</main>

<%- include("partials/footer") %>
```

- `profile.phone || "Not added"` - fallback text for an empty phone.
- `new Date(profile.created_at).toDateString()` - you can run normal JavaScript inside `<%= %>`. This turns the date into something like `Sun Nov 08 2026`.
- Each booking has a **Release space** form that posts to `/spaces/:id/release`.

Step 11 - 404.ejs and error.ejs

views/404.ejs

```
<%- include("partials/header", { title: "Not found" }) %>

<main>
    <div class="card">
        <h1>404 - Page not found</h1>
        <p>The page you are looking for does not exist.</p>
        <a class="btn" href="/">Go home</a>
    </div>
</main>

<%- include("partials/footer") %>
```

views/error.ejs

```
<%- include("partials/header", { title: "Error" }) %>
```

```
<main>
    <div class="alert-error">
        <h2>Oops!</h2>
        <p><%= message %></p>
    </div>
    <a class="btn" href="/">Go home</a>
</main>

<%- include("partials/footer") %>
```

`message` comes from the error handler in part 13: `res.status(500).render("error", { message: ... })`.

Step 12 - Run and test

Start MySQL, then `npm run dev`. The terminal should show:

```
Server running at http://localhost:3000
Database connected successfully
```

Test checklist

- Open `http://localhost:3000/dashboard` while logged out - you should land on the **login** page.
- Open `http://localhost:3000/abc` - you should see the **404** page.
- Sign up with passwords that do not match - red error, and your name/email are still filled in.
- Sign up properly - you land on login with the green "Account created" message.
- Sign up again with the same email - "That email is already registered".
- Log in with a wrong password - "Wrong email or password".
- Log in correctly - you see the dashboard with 3 parking lots.
- While logged in, open `/login` - you are sent back to the dashboard.
- Open a lot and book a space - it now says **Yours**, and the free count on the dashboard goes down.
- Go to Profile - your booking is listed. Release it - it is gone, and the space is free again.
- In phpMyAdmin, look at the `users` table - the password column shows a hash starting with `$2b$`, not your real password.
- Log out, then press the browser Back button and refresh - you are sent to login.

Common errors and fixes

You see	Why	Fix
`ECONNREFUSED ...3306`	MySQL is not running	Start MySQL in XAMPP / Services
`Unknown database 'parkease_db'`	Tables not created	Run `database.sql`
`Access denied for user 'root'`	Wrong DB login	Fix `DB_USER` / `DB_PASSWORD` in `.env`
`Failed to lookup view "x"`	File name typo	`res.render("x")` needs `views/x.ejs`
`error is not defined` (EJS)	Route did not send it	Add it to the `res.render` object
`req.body` is undefined	Form middleware missing	Keep `express.urlencoded` above routes
Page with no styling	Wrong CSS path	Use `/css/style.css` with a leading `/`
Page loads forever	Middleware never calls `next()`	Every path must `next()`, redirect or render

`EADDRINUSE :::3000`	Server already running	Close the other terminal or change `PORT`

What changed from your original server.js

- `mysql2` became `mysql2/promise`, so `await` works with the database.
- Database and session settings moved to `.env`.
- `connection.release()` added after the connection test.
- `isAuthenticated` is now actually used on dashboard, parking and profile. `isGuest` added for login/signup.
- POST routes added for signup, login, logout, book and release.
- `/parking` became `/parking/:id` so it knows which lot to show.
- `res.locals.user` added so the nav bar knows who is logged in.
- 404 page and error handler added at the bottom.

Tip: Next ideas once this works: let users edit their profile, add a bookings history table with start and end times, or add an admin page to create new parking lots.